

Kompletní průvodce instalací a konfigurací NIDS na Raspberry Pi

Tato dokumentace popisuje proces tvorby síťového detekčního systému na platformě Raspberry Pi 5 (architektura ARM64) s operačním systémem Ubuntu Server 24.04 LTS. Příprava, instalace a konfigurace všech potřebných komponent systému, trénování a nasazení modelu strojového učení.

Senzor: Zeek (bare-metal instalace)

Transport: Telegraf (bare-metal instalace)

Ukládání a vizualizace: InfluxDB a Grafana (Docker kontejnery)

Detekční engine: Vlastní Python skript s modely strojového učení (bare-metal)

Pro přenos souborů mezi klientskou stanicí (Windows) a Raspberry Pi byl využit program **WinSCP**.

1. Příprava základního systému a zabezpečení

Konfigurace časového pásma a síťových nastavení:

```
sudo timedatectl set-timezone Europe/Prague
sudo nano /etc/netplan/50-cloud-init.yaml

# Nahraďte obsah souboru následující konfigurací (přizpůsobte IP adresy
všech konfigurací Vaší síti a potřebám):

network:
  version: 2
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 192.168.69.6/24
      routes:
        - to: default
          via: 192.168.69.254
      nameservers:
        addresses: [192.168.69.254, 1.1.1.1]
```

Aplikujte síťové nastavení:

```
sudo netplan apply
```

Vytvoření uživatele a nastavení přístupových práv:

```
# Vytvoření dedikovaného uživatele pro správu systému
sudo adduser klugy
# Přidání uživatele do skupiny sudo pro možnost eskalace privilegií
sudo usermod -aG sudo klugy
```

Zabezpečení přístupu pomocí SSH klíčů: Pro zvýšení bezpečnosti bylo zakázáno přihlašování heslem a byl implementován přístup pouze pomocí SSH klíčů.

Na klientském PC (Windows): Pomocí nástroje **PuTTYgen** byl vygenerován pár klíčů (typ RSA).

Export klíčů: Privátní klíč byl uložen na klientském PC (formát .ppk pro PuTTY/WinSCP). Veřejný klíč byl zkopírován do schránky.

Na Raspberry Pi: Byl vytvořen adresář pro klíče a veřejný klíč byl do něj vložen:

```
# Vytvoření skryté složky pro SSH konfiguraci s omezeným přístupem
mkdir -p /home/klugy/.ssh
chmod 700 /home/klugy/.ssh

# Vytvoření souboru pro povolené klíče
nano /home/klugy/.ssh/authorized_keys
# (Do souboru byl vložen řetězec veřejného klíče z PuTTYgen)

# Omezení práv pouze na vlastníka a změna vlastnictví
chmod 600 /home/klugy/.ssh/authorized_keys
sudo chown -R klugy:klugy /home/klugy/.ssh
```

Zákaz přihlašování heslem:

```
sudo nano /etc/ssh/sshd_config.d/50-cloud-init.conf
# V souboru změnit yes na no

PasswordAuthentication no
```

Restart služby SSH:

```
sudo systemctl restart ssh
```

2. Instalace kontejnerizačního nástroje Docker

Systém využívá Docker pro izolaci a snadnou správu databáze a vizualizační vrstvy.

```
# Aktualizace systémových balíčků a instalace základních nástrojů
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget gnupg2 ca-certificates apt-transport-https
software-properties-common

# Stažení a import ověřovacího GPG klíče Dockeru pro bezpečné stahování
balíčků
sudo install -m 0.755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Přidání repozitáře s vynucením nativní architektury procesoru
echo \
  "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalace jádra Dockeru a modulu Compose pro orchestraci kontejnerů
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

3. Sběr dat: Instalace a nastavení Zeek

Z důvodu chybějících balíčků pro Ubuntu 24.04 (ARM64) v repozitářích tvůrců, byl pro instalaci Zeeku úspěšně využit repozitář pro starší verzi Ubuntu 22.04, jehož binární soubory jsou plně kompatibilní.

```
# Přidání repozitáře openSUSE Build Service (využita kompatibilní verze
22.04 pro ARM64)
echo 'deb [arch=arm64]
http://download.opensuse.org/repositories/security:zeek/xUbuntu_22.04/
/' | sudo tee /etc/apt/sources.list.d/security:zeek.list
curl -fsSL
https://download.opensuse.org/repositories/security:zeek/xUbuntu_22.04/R
elease.key | gpg --dearmor | sudo tee
/etc/apt/trusted.gpg.d/security_zeek.gpg > /dev/null

# Instalace verze s dlouhodobou podporou (LTS) a nástroje pro ACL práva
sudo apt update
sudo apt install -y zeek-lts acl

# Zajištění dostupnosti příkazů Zeeku v globálním kontextu terminálu
echo 'export PATH=$PATH:/opt/zeek/bin' >> /root/.bashrc
source /root/.bashrc
```

Konfigurace Zeeku:

```
sudo nano /opt/zeek/etc/node.cfg
# Do souboru vložit následující:
# Definuje, ze které síťové karty má Zeek zachytávat provoz.

[zeek]
type=standalone
host=localhost
interface=eth0 # Název interface se může lišit
```

```
sudo nano /opt/zeek/etc/networks.cfg
# Přidejte rozsah Vaší lokální sítě a název:
192.168.69.0/24          Klugy IP space
```

JSON výstup a MAC adresy:

Výchozí logy Zeeku jsou tabulkové (TSV) a neobsahují L2 hlavičky. Následující skripty vynucují formát JSON a extrakci MAC adres.

```
sudo nano /opt/zeek/share/zeek/site/local.zeek
# Přidejte tyto dva řádky na úplný konec souboru

redef LogAscii::use_json = T;
@load policy/protocols/conn/mac-logging
```

```
sudo nano /opt/zeek/etc/zeekctl.cfg

# Změňte hodnotu LogRotationInterval na 86400 (1 den)
```

Vypnutí optimalizace síťové karty (NIC Offloading):

Moderní síťové karty manipulují s pakety (např. slučování) pro úsporu výkonu procesoru. Pro správnou funkci NIDS je to však nežádoucí, jelikož systém vyžaduje vidět provoz v původním stavu.

```
sudo nano /etc/systemd/system/zeek-nic-setup.service

[Unit]
Description=Disable NIC offloading for Zeek
After=network.target

[Service]
Type=oneshot
# Příkaz ethtool vypíná Generic Receive Offload, Generic Segmentation
Offload a TCP Segmentation Offload
ExecStart=/usr/sbin/ethtool -K eth0 gro off gso off tso off
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Nasazení Zeeku a spuštění služeb:

```
sudo systemctl enable --now zeek-nic-setup.service

# Instalace vytvoří adresářovou strukturu a zkompiluje skripty
sudo /opt/zeek/bin/zeekctl install
# Deploy aplikuje nastavení a spustí proces odposlouchávání
sudo /opt/zeek/bin/zeekctl deploy
```

4. Databázové vrstva a vizualizace

Systém InfluxDB a Grafana jsou spouštěny pomocí Docker Compose.

Příprava adresářů:

```
# Vytvoření složek na hostiteli, aby data přežila restart kontejnerů
sudo mkdir -p /opt/docker/influxdb/data
sudo mkdir -p /opt/docker/influxdb/config
# Oprávnění pro lokálního uživatele, aby služba InfluxDB mohla zapisovat
sudo chown -R klugy:klugy /opt/docker/influxdb

sudo mkdir -p /opt/docker/grafana/data
# GID 472 je standardní identifikátor uživatele v obrazu Grafany
sudo chown -R 472:472 /opt/docker/grafana/data

mkdir -p ~/monitoring && cd ~/monitoring
nano docker-compose.yml
```

Obsah docker-compose.yml:

```
services:
  influxdb:
    image: influxdb:latest
    container_name: influxdb
    restart: unless-stopped # Kontejner se sám zapne po restartu RPi
    ports:
      - "8086:8086"
    volumes:
      - /opt/docker/influxdb/data:/var/lib/influxdb2
      - /opt/docker/influxdb/config:/etc/influxdb2
    environment:
      # Automatické vytvoření organizace a bucketu při prvním spuštění
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=*username*
      - DOCKER_INFLUXDB_INIT_PASSWORD=*password*
      - DOCKER_INFLUXDB_INIT_ORG=home # možné zvolit vlastní
      - DOCKER_INFLUXDB_INIT_BUCKET=zeek # možné zvolit vlastní

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    volumes:
      - /opt/docker/grafana/data:/var/lib/grafana
```

```
depends_on:
  - influxdb # Zajišťuje start Grafany až po nastartování databáze
environment:
  - GF_PLUGIN_ALLOW_LOCAL_MODE=true
  - GF_SERVER_DOMAIN=192.168.69.6
  - GF_SERVER_ROOT_URL=http://192.168.69.6:3000/
```

Spuštění infrastruktury:

```
sudo docker compose up -d
```

Po spuštění se přihlaste do InfluxDB (<http://192.168.69.6:8086>) a v sekci **Load Data -> API Tokens** vygenerujte **All Access Token**.

5. Telegraf (Přenos událostí)

Telegraf funguje jako agent na hostitelském stroji čtoucí JSON logy a přeposílající je do InfluxDB. Byla využita předkompilovaná binární verze.

Stažení a instalace:

```
cd /opt/  
sudo mkdir telegraf && cd telegraf  
sudo wget  
https://dl.influxdata.com/telegraf/releases/telegraf-1.37.3_linux_arm64.  
tar.gz  
sudo tar xf telegraf-1.37.3_linux_arm64.tar.gz  
sudo mkdir -p /etc/telegraf
```

Konfigurace Telegrafu:

```
sudo nano /etc/telegraf/telegraf.conf  
  
[agent]  
interval = "10s" # Jak často Telegraf agreguje metriky  
flush_interval = "10s" # Jak často data odesílá do databáze  
  
[[outputs.influxdb_v2]]  
urls = ["http://127.0.0.1:8086"]  
token = "<VÁŠ_TOKEN>"  
organization = "home"  
bucket = "zeek"  
  
[[inputs.tail]]  
# Seznam sledovaných výstupů Zeeku  
files = [  
    "/opt/zeek/spool/zeek/conn.log",  
    "/opt/zeek/spool/zeek/dns.log",  
    "/opt/zeek/spool/zeek/http.log",  
    "/opt/zeek/spool/zeek/ssl.log",  
    "/opt/zeek/spool/zeek/files.log",  
    "/opt/zeek/spool/zeek/known_hosts.log",  
    "/opt/zeek/spool/zeek/known_services.log",  
    "/opt/zeek/spool/zeek/weird.log",  
    "/opt/zeek/spool/zeek/ssh.log",  
    "/opt/zeek/spool/zeek/capture_loss.log",  
    "/opt/zeek/spool/zeek/ntp.log",  
    "/opt/zeek/spool/zeek/quic.log"  
]
```

```
# Zamezení přetížení DB čtením starých logů při restartu
from_beginning = false
data_format = "json"
json_time_key = "ts"
json_time_format = "unix"
# Definice indexovatelných prvků (Tags) pro rychlé vyhledávání v
databázi
tag_keys = ["id.orig_h", "id.resp_h", "proto", "query", "service",
"orig_l2_addr", "resp_l2_addr"]
```

Zeek vytváří logy pod uživatelem root. Aby je mohl proces telegraf číst bez narušení bezpečnostního modelu (bez nutnosti plných root práv), jsou využity seznamy řízení přístupu (ACL), které dávají Telegrafu výlučné právo na čtení.

```
# Aplikace práv na stávající logy (rx = čtení a přístup složkou)
sudo setfacl -R -m u:klugy:rx /opt/zeek/logs/
sudo setfacl -R -m u:klugy:rx /opt/zeek/spool/zeek/
# Aplikace dědičnosti (d) pro logy vytvořené v budoucnu
sudo setfacl -R -d -m u:klugy:rx /opt/zeek/logs/
sudo setfacl -R -d -m u:klugy:rx /opt/zeek/spool/zeek/
```

Vytvoření Systemd služby pro Telegraf:

```
sudo nano /etc/systemd/system/telegraf.service

[Unit]
Description=Telegraf Service
After=network.target

[Service]
# Exekuce binárního souboru s konkrétní konfigurací
ExecStart=/opt/telegraf/telegraf-1.37.3/usr/bin/telegraf --config
/etc/telegraf/telegraf.conf
Restart=on-failure
User=root

[Install]
WantedBy=multi-user.target
```

Aktivace a spuštění:

```
sudo systemctl daemon-reload
sudo systemctl enable --now telegraf
```

6. Custom ML Detection Engine (Strojové učení)

Příprava virtuálního Python prostředí:

```
# Vytvářet bez sudo, jinak budou složky uživatele vlastněny rootem.

mkdir -p /home/klugy/MAC_detekce /home/klugy/ARCHIVE
cd /home/klugy/MAC_detekce
sudo apt install -y python3-pip python3.12-venv
# Inicializace izolovaného prostředí pro zajištění compatibility
# knihoven
python3 -m venv venv
source venv/bin/activate
pip install influxdb-client pandas scikit-learn numpy joblib
```

Proces trénování modelů `model_home.pkl` a `model_away.pkl` vyžaduje historická provozní data. Prostřednictvím protokolu SFTP (WinSCP) byly dříve zachycené `.log` soubory nakopírovány do cesty `/home/klugy/ARCHIVE`.

Příkazy pro spuštění tréninku:

```
cd /home/klugy/MAC_detekce
source venv/bin/activate
# Vytvoření dočasné zálohy aktuálních dat do ARCHIVU
cp -r /opt/zeek/logs/* /home/klugy/ARCHIVE/
# Exekuce vyhodnocovacího skriptu
python3 train_network_model_v5.py
```

Výstupem tohoto procesu jsou natrénované modely (`model_home.pkl`, `model_away.pkl`) uložené v pracovní složce.

Nasazení detekční služby:

```
[Unit]
Description=Zeek ML Real-time Network Monitor
After=network.target influxdb.service zeek.service

[Service]
Type=simple
# Skript beží pod uživatelem root, aby měl garantovaný přístup ke složce
# /opt/zeek/spool/
# Skript je nutné spouštět uvnitř virtuálního prostředí (venv)
ExecStart=/home/klugy/MAC_detekce/venv/bin/python
/home/klugy/MAC_detekce/realtime_ml_monitor.py
WorkingDirectory=/home/klugy/MAC_detekce
StandardOutput=inherit
```

```
StandardError=inherit
Restart=always
User=root
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

Aktivace a spuštění monitoringu:

```
sudo systemctl daemon-reload
sudo systemctl enable --now zeek-ml-monitor.service
```